

Esercitazione

Connessioni TCP

Visualizzare tutte le connessioni TCP in stato **ESTAB** e stampare **solo** l'IP remoto (senza porta).

Esempio:

Da 192.168.1.10:45678 deve uscire 192.168.1.10

Suggerimento: usare `ss -tn` e `cut -d:`

Esercitazione

Connessioni TCP

Visualizzare tutte le connessioni TCP in stato **ESTAB** e stampare **solo** l'IP remoto (senza porta).

Esempio:

Da 192.168.1.10:45678 deve uscire 192.168.1.10

Suggerimento: usare `ss -tn` e `cut -d:`

Soluzione:

```
ss -tn | grep ESTAB | awk '{print $5}' | cut -d: -f1
```

Esercitazione

Interfacce

Usando una pipeline di comandi UNIX, estrarre e stampare **solo i nomi delle interfacce di rete** presenti sulla macchina.

Esempio:

lo

eth0

enp0s3

wlan0

docker0

Suggerimento: usare `ip link show | grep '^ ...'`

Esercitazione

Interfacce

Usando una pipeline di comandi UNIX, estrarre e stampare **solo i nomi delle interfacce di rete** presenti sulla macchina.

Esempio:

lo

eth0

enp0s3

wlan0

docker0

Suggerimento: usare `ip link show | grep '^ ...'`

Soluzione:

```
ip link show | grep '^ [0-9]' | cut -d: -f2 | sed 's/^ //'
```

Esercitazione

Interfacce

Usando una pipeline di comandi UNIX, estrarre e stampare **solo i nomi delle interfacce di rete** presenti sulla macchina che non sono UP

Esempio:

lo

eth0

enp0s3

wlan0

docker0

Suggerimento: usare `ip link show | grep '^ ...' | grep ...`

Esercitazione

Interfacce

Usando una pipeline di comandi UNIX, estrarre e stampare **solo i nomi delle interfacce di rete** presenti sulla macchina che non sono UP

Esempio:

lo

eth0

enp0s3

wlan0

docker0

Suggerimento: usare `ip link show | grep '^ ...' | grep ...`

Soluzione:

```
ip link show | grep '^ [0-9]' | grep -v UP | cut -d: -f2 | sed 's/^ //'
```

Esercitazione

Monitoraggio di file in una directory

- Scrivere uno script Bash che:
 - Riceve come argomento una directory e una stringa
 - Ogni 5 secondi controlla se ci sono file .txt che hanno almeno una occorrenza della stringa
 - Se il numero cambia rispetto alla volta precedente, stampa un messaggio sullo schermo

Esercitazione

Monitoraggio di file in una directory

```
#!/bin/bash
# TODO: controllare che ci siano esattamente 2 argomenti
# TODO: controllare che DIR sia una directory valida
old_count=-1 # numero precedente di file che contengono STR
while true; do
    count=0
    # Scorro tutti i file .txt nella directory
    for f in "$DIR"/*.txt; do
        # TODO: verificare che f sia davvero un file regolare
        # TODO: usare grep -q per controllare se il file contiene STR
        # se sì, incrementare count
    done
    # TODO: se count è diverso da old_count, stampare un messaggio
    # e aggiornare old_count
    sleep 5
done
```

Esercitazione

Sostituzione con sed basata sul nome del file

- Scrivere uno script Bash che:
 - Riceve come unico argomento un file di testo.
 - Prende la prima lettera e l'ultima lettera del nome del file.
 - In quel file, sostituisce tutte le parole che:
 - iniziano con la prima lettera
 - e finiscono con l'ultima lettera del nome del file stesso.
- Esempio:
 - nel file chiamato **esempio**, le parole: empio, egitto, editto sono sostituite con esempio

Esercitazione

Sostituzione con sed basata sul nome del file

```
#!/bin/bash
# TODO: controllare che sia stato passato un argomento
# TODO: controllare che il file esista
# Ricavare solo il nome del file (senza percorso)
BASENAME=$(basename "$FILE")

# TODO: estrarre la prima lettera di BASENAME (usare cut)
# FIRST=$( ... )
# TODO: estrarre l'ultima lettera di BASENAME (usare cut)
# LAST=$( ... )

# TODO:
# tutte le parole che iniziano con FIRST e finiscono con LAST
# con il nome del file BASENAME.
# Suggerimento: usare i delimitatori di parola POSIX \< e \>
#
```

Esercitazione

Conta i thread (LWP) per ogni processo dell'utente

- Scrivere uno script Bash che:
 - Riceve come argomenti uno **username** e una soglia **N** (intero)
 - Usa il comando: `ps -L -u <utente> -o pid,lwp --no-headers` per elencare i thread (LWP)
 - Utilizzando `awk`, deve:
 - contare, per ogni PID, quanti LWP (thread) ha;
 - stampare solo i processi che hanno più di N LWP.
- Deve stampare il numero totale di processi che superano la soglia.

Processi dell'utente alice con più di 10
LWP:

PID 1234 → 23 LWP

PID 5678 → 42 LWP

Totale processi sopra soglia: 2

Esercitazione

Conta i thread (LWP) per ogni processo dell'utente

```
#!/bin/bash
# Controllo argomenti
USER_NAME="$1"
THRESHOLD="$2"
ps -L -u "$USER_NAME" -o pid,lwp --no-headers | awk -v soglia="$THRESHOLD" '
BEGIN {
    # inizializzazione contatore processi
}
{
    pid = $1;
    # TODO: incrementare il numero di LWP per questo pid
}
END {
    # TODO: scorrere tutti i pid nell array count
    # e stampare solo quelli con count[pid] > soglia
    # per ogni processo sopra soglia, incrementare proc_over
    # TODO: alla fine stampare il totale:
}'
```

Esercitazione

Esercizio: fork, printf

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("A");
    fork();
    printf("B\n");
    return 0;
}
```

Cosa stampa questo programma? Perché?

Cosa accade se al posto di `printf("x");` usiamo `write(1, "x", 1);` ?

Esercitazione

Esercizio: cond wait

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
```

```
int ready = 0;
```

```
void* worker(void* arg) {  
    int id = (int)(long)arg;  
    pthread_mutex_lock(&m);  
    if (!ready)  
        pthread_cond_wait(&c, &m);  
    printf("Thread %d entra nella sezione critica\n", id);  
    // questo thread "consuma" la condizione e la rimette a 0  
    ready = 0;  
    pthread_mutex_unlock(&m);  
    return NULL;  
}
```

Cosa stampa questo programma?

Quando i thread entrano i sezione critica quanto vale ready?

Esercitazione

Esercizio: cond wait error

```
int main(void) {  
    pthread_t t1, t2;  
  
    pthread_create(&t1, NULL, worker, (void*)1);  
    pthread_create(&t2, NULL, worker, (void*)2);  
  
    sleep(1);  
  
    pthread_mutex_lock(&m);  
    ready = 1;  
    pthread_cond_broadcast(&c); // sveglio TUTTI i thread che aspettano  
    pthread_mutex_unlock(&m);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
    return 0;  
}
```

Esercitazione

Esercizio: fork, pipe e select

```
#define BUF 4
```

```
int main(void) {  
    int fd[2]; char buf[BUF];  
    pipe(fd);  
  
    if (fork() == 0) {  
        close(fd[1]); int p = fd[0];  
        while (1) {  
            fd_set s;  
            FD_ZERO(&s); FD_SET(0, &s); FD_SET(p, &s);  
            int m = p > 0 ? p : 0;  
            if (select(m + 1, &s, NULL, NULL, NULL) <= 0) break;  
            if (FD_ISSET(p, &s)) {  
                int n = read(p, buf, BUF - 1);  
                if (n <= 0) break;  
                buf[n] = '\0'; printf("padre: %s ", buf);  
            }  
        }  
    }  
}
```

Lanciare il programma **es_fork_pipe_select.c** e provare a scrivere delle stringhe.

Cosa succede? Perché?

Esercitazione

```
if (FD_ISSET(0, &s)) {
    int n = read(0, buf, BUF - 1);
    if (n <= 0) break;
    buf[n] = '\0'; printf("figlio: %s ", buf);
}
fflush(stdout);
}
return 0;
} else {
    close(fd[0]);
    int p = fd[1];
    while (1) {
        fd_set s; FD_ZERO(&s); FD_SET(0, &s);
        if (select(1, &s, NULL, NULL, NULL) <= 0) break;
        if (FD_ISSET(0, &s)) {
            int n = read(0, buf, BUF - 1); if (n <= 0) break;
            buf[n] = '\0'; write(p, buf, n);
        }
    }
    close(p); wait(NULL);
}
}
```

Esercitazione

Esercizio: client multi-thread errore

```
void* sender(void* arg) {  
    const char *msgA = "A";  
    const char *msgB = "B";  
  
    pthread_mutex_lock(&mtx);  
  
    sendto(sockfd, msgA, strlen(msgA), 0,  
           (struct sockaddr*)&servaddr, sizeof(servaddr));  
    printf("sender: inviato A\n");  
  
    sendto(sockfd, msgB, strlen(msgB), 0,  
           (struct sockaddr*)&servaddr, sizeof(servaddr));  
    printf("sender: inviato B\n");  
  
    pthread_mutex_unlock(&mtx);  
    return NULL;  
}
```

Lanciare il programma
es_multi_client_err.c e
es_server.c

Qual è il problema?
Come si risolve?

Esercitazione

Esercizio: client multi-thread errore

```
void* receiver(void* arg) {  
    char buf[64];  
  
    pthread_mutex_lock(&mtx);  
  
    ssize_t n = recvfrom(sockfd, buf, sizeof(buf)-1, 0, NULL, NULL);  
    if (n > 0) {  
        buf[n] = '\0';  
        printf("receiver: ricevuto <%s>\n", buf);  
    } else {  
        perror("recvfrom");  
    }  
  
    pthread_mutex_unlock(&mtx);  
    return NULL;  
}
```

Esercitazione

Esercizio: client multi-thread errore

```
int main(void) {  
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);  
  
    memset(&servaddr, 0, sizeof(servaddr));  
    servaddr.sin_family = AF_INET;  
    servaddr.sin_port = htons(5000);  
    inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);  
  
    pthread_t tsend, trecv;  
  
    pthread_create(&tsend, NULL, sender, NULL);  
    pthread_create(&trecv, NULL, receiver, NULL);  
  
    pthread_join(tsend, NULL);  
    pthread_join(trecv, NULL);  
    close(sockfd);  
    return 0;  
}
```

Esercitazione

Esercizio: client multi-thread soluzione

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

```
int turno = 0; // 0 = tocca al sender, 1 = tocca al receiver
```

```
void* sender(void* arg) {
```

```
    const char *msg1 = "ciao";
```

```
    const char *msg2 = "mondo";
```

```
    pthread_mutex_lock(&mtx);
```

```
    while (turno != 0)
```

```
        pthread_cond_wait(&cond, &mtx);
```

```
    // SEZIONE CRITICA -> serve tenere i lock durnate il send?
```

```
    turno = 1;
```

```
    pthread_cond_signal(&cond);
```

```
    pthread_mutex_unlock(&mtx);
```

```
    return NULL;
```

```
}
```

Esercitazione

Esercizio: client multi-thread soluzione

```
void* receiver(void* arg) {
    char buf[256];

    pthread_mutex_lock(&mtx);
    while (turno != 1)
        pthread_cond_wait(&cond, &mtx);
    // ora è il turno del ricevitore
    pthread_mutex_unlock(&mtx);

    // I/O di rete senza tenere il mutex
    ssize_t n = recvfrom(sockfd, buf, sizeof(buf)-1, 0, NULL, NULL);
    if (n > 0) {
        buf[n] = '\0';
        printf("receiver: ricevuto <%s>\n", buf);
    } else { perror("recvfrom"); }
    return NULL;
}
```

Esercitazione

Esercizio: lancio di N client in background

```
#!/bin/bash
```

```
N=$1 # numero di client da lanciare
```

```
for ((i=1; i<=N; i++)); do
```

```
    ./client_program &
```

```
done
```

```
wait # attende che tutti i client finiscano
```

Perché può
dare problemi?

Esercitazione

Esercizio — Thread che si svegliano a cascata

- Scrivere un programma C che:
 - crea **3 thread**: t1, t2, t3;
 - l'esecuzione deve avvenire **in ordine**:
 - t1 stampa:
 - "t1 inizia"
 - "t1 finisce → sveglia t2"
 - t2 **deve aspettare** che t1 lo svegli, poi stampa:
 - "t2 inizia"
 - "t2 finisce → sveglia t3"
 - t3 **deve aspettare** che t2 lo svegli, poi stampa:
 - "t3 inizia"
 - "t3 finisce«
- Per la sincronizzazione, usare:
 - una var. glob. int stato per il turno, pthread_mutex_t, una pthread_cond_t

Esercitazione

Esercizio — Thread che si svegliano a cascata

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

```
int stato = 1; // 1 = tocca a t1, 2 = t2, 3 = t3
```

```
void* thread1(void* arg) {  
    //TODO: mutex  
    while (stato != 1) {  
        pthread_cond_wait(&cond, &mtx);  
    }  
    printf("t1 inizia\n");  
    printf("t1 finisce -> sveglia t2\n");  
    // TODO: aggiornare stato in modo che tocchi a t2  
    //TODO: mutex  
    return NULL;  
}
```

Esercitazione

Esercizio — Thread che si svegliano a cascata

```
int main(void) {  
    pthread_t t1, t2, t3;  
  
    // TODO: creare i 3 thread  
    // pthread_create(&t1, ...)  
  
    // TODO: fare la join dei 3 thread  
  
    // TODO: distruggere mutex e condition  
  
    return 0;  
}
```

Esercitazione

Esercizio — Padre che scrive al figlio e figlio con select() su tastiera

- Scrivere un programma C che:
 - Crea una **pipe**
 - Esegue una fork() per creare un processo figlio
 - Il processo padre invia 10 stringhe al figlio via pipe ad intervalli random
 - Il figlio usa select() per **attendere contemporaneamente** i dati dal padre via socket e dall'utente da tastiera
 - Se arriva dal padre scrive: "[PADRE] <contenuto>«
 - Se arriva dal tastiera scrive: "[UTENTE] <contenuto>"

Esercitazione

Esercizio — Padre che scrive al figlio e figlio con select() su tastiera

```
int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); exit(EXIT_FAILURE);}

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(EXIT_FAILURE); }

    if (pid == 0) {
        /* FIGLIO */
        int pipe_fd = fd[0];
        int pipe_closed = 0;
        char buf[MAX_BUF];

        while (1) {
            fd_set readfds;
            FD_ZERO(&readfds);
            // TODO: aggiungere pipe_fd al set
            // TODO: aggiungere stdin (fd 0) al set
```